

Corporate Learning Path Recommender

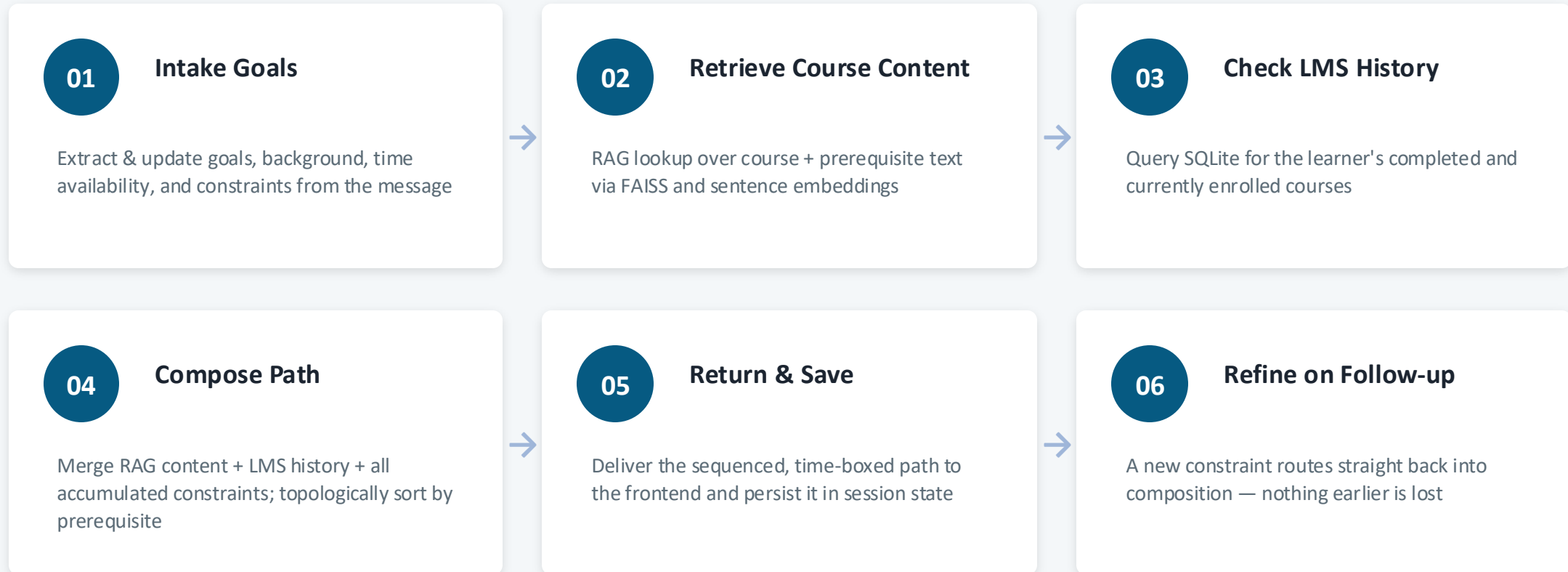
A context-aware agent that turns career goals, current skills, and LMS history into a sequenced, time-boxed course plan

THE PROBLEM

A corporate training provider offers hundreds of courses across SAP, Workday, Salesforce, Cloud, AI, and Cybersecurity tracks. Learners rarely know which sequence fits their goals, current skill level, and already-completed work — and a static catalog can't hold a conversation, remember what was said three turns ago, or replan when a new constraint shows up.

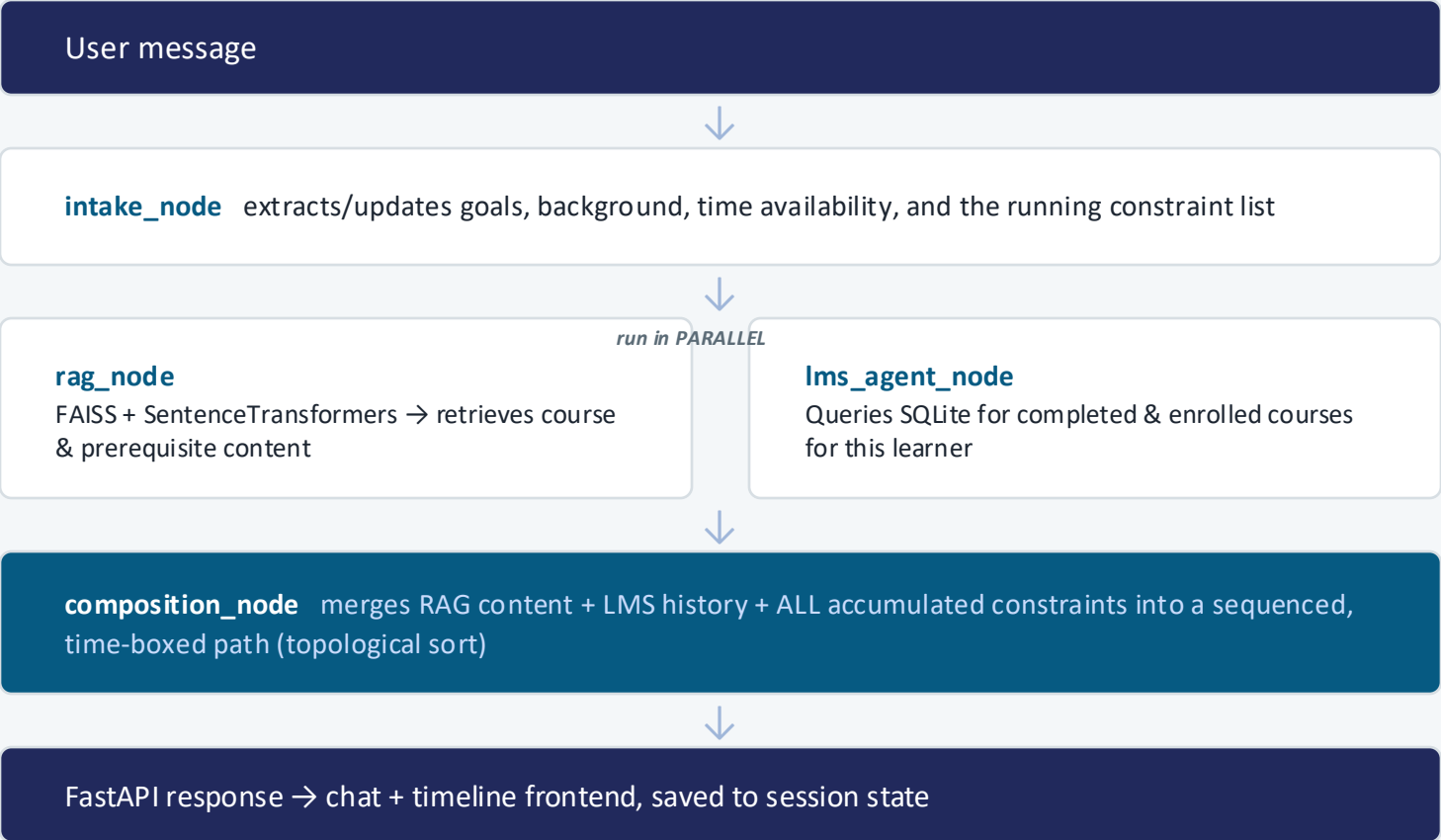
What the System Does

End-to-end flow, from a learner's first message to a refined, time-boxed path



Architecture

Two data sources are fetched in parallel and converge before the path is composed



ON A REFINEMENT TURN

The graph skips intake’s parallel fetch entirely. A new constraint routes directly back into composition_node, reusing the stored RAG context and LMS history alongside every earlier constraint — so nothing stated in turn 1 is forgotten by turn 4.

intake → composition
(bypasses rag & lms_tool)

Context-Aware Refinement: Compose, Then Accumulate

Every follow-up turn routes back into `composition_node` — no earlier constraint is ever dropped

1

First turn — full pipeline

intake → rag_node + lms_tool_node (parallel) → composition_node produces the initial sequenced path



2

Every turn after — direct reroute

New constraint goes straight to `composition_node`, reusing stored RAG context, LMS history, and every prior constraint in state

Prerequisite-aware & constraint-aware: `composition_node` topologically sorts by prerequisite and marks a superseded constraint explicitly (`superseded: true`) rather than silently discarding it

REAL EXAMPLE — FROM ACTUAL TEST RUN

Turn 1 (Goal): “Wants Salesforce admin” → agent recommends a 6-course path starting with foundational SF-101

Turn 2 (Constraint): “No prior experience” → path keeps the foundational start; constraint stored

Turn 3 (New constraint): “Only 5 hours/week” → timeline recalculates across more weeks, while retaining the Turn 2 constraint

Turn 4 (Revision): “Actually I have Salesforce basics” → Turn 2 constraint marked `superseded: true`; active constraints and recommendations update

Tech Stack & Why We Chose It

Component	Choice	Why
Agent orchestration	LangGraph	Manages the AI workflow, connects all steps, and maintains context during follow-up conversations.
Vector search	FAISS + sentence-transformers (all-MiniLM-L6-v2)	Helps with semantic search, fast, fully local, no API cost for embedding course content
Database	SQLite + SQLAlchemy ORM	Lightweight LMS store for enrollment/completion history — easy to seed and reset for repeatable testing
API framework	FastAPI	Chat endpoint + session state, with interactive docs for demoing and testing every endpoint
Tool calling	Native LLM tool calling, deterministic fallback	USE_REAL_TOOL_CALLING flag lets the LMS lookup run as an LLM-driven tool call or a plain node, so the system stays robust across model gateways

Trade-offs & Design Decisions

Honest accounting of what each choice cost us

Decision	What we gained	What we traded away
Refinement re-enters at composition_node, not intake	Prior constraints and retrieved context are never dropped on follow-up turns	Extra state-management complexity — the graph must track and merge a running constraint list, not just the latest message
Deterministic lms_tool_node with optional native tool calling	A robust default that works without a tool-calling-capable model, plus an upgrade path when one is available	Two code paths to maintain and test instead of one
Parallel RAG + LMS fetch	Faster response — course content and learner history retrieved concurrently	composition_node must reconcile two independent data sources rather than reasoning over one at a time
Synthetic 50-course catalog & 5 learner profiles	Full control to test deep prerequisite chains (depth 4) and edge cases like completed-vs-enrolled overlap	Doesn't prove robustness against messy, inconsistent real-world corporate LMS data

Results & What This Demonstrates

RAG Retrieval Quality

96.7% answer relevancy and 97.2% context recall (RAGAS-style)
across 12 structured course/prerequisite Q&A pairs

Recommendation Relevance

92.0% average path overlap against 5 expert reference personas;
100% correct prerequisite ordering — 0 violations

Context-Aware Refinement

4-turn scripted conversation validates constraint accumulation,
retention, and explicit supersession across the session

Automated Testing

26 automated unit tests pass in ~2 seconds, covering composition,
LMS filtering, and refinement logic